Донецький національний технічний університет



Тема 1. Вступ до DevOps. Лекція 1. Вступ до дисципліни

Предмет: Технології DevOPS
Розділ: Кібербезпека
Галузь: Інформаційні технології

Укладач: проф. Дорогий Я.Ю.



Лекційні питання

- Мета та структура курсу
- Вступ в DevOps
- Continuous Integration
- Середовища
- Continuous Delivery та Deployment
- Continuous Testing
- Метрики

Мета курсу

- Метою дисципліни "Технології DevOps" є формування системи теоретичних та практичних знань та навичок щодо процесу швидкого розгортання, супроводу, моніторингу та загальної автоматизації певних задач в різних середовищах на Linux-сумісних платформах.

Очікувані результати

Внаслідок вивчення курсу студенти повинні вміти:

- ❑застосовувати отримані навики самостійного вивчення навчальної та наукової літератури, володіти понятійним апаратом;
- ❑застосовувати систему контролю версій Git;
- ❑використовувати Bash та Python для автоматизації задач;
- ❑розгортати, налаштовувати та використовувати інструментарій віртуалізації Vagrant та систему розгортання Capistrano;
- ❑використовувати систему контейнеризації Docker;
- ❑розгортати, налаштовувати та використовувати системи управління конфігураціями Chef, Puppet та Ansible;
- ❑розгортати, налаштовувати та використовувати систему безперервної інтеграції Jenkins;
- ❑розгортати, налаштовувати та використовувати систему оркестрації Kubernetes;
- ❑розгортати, налаштовувати та використовувати системи моніторингу та налаштування сервісів.

Структура курсу

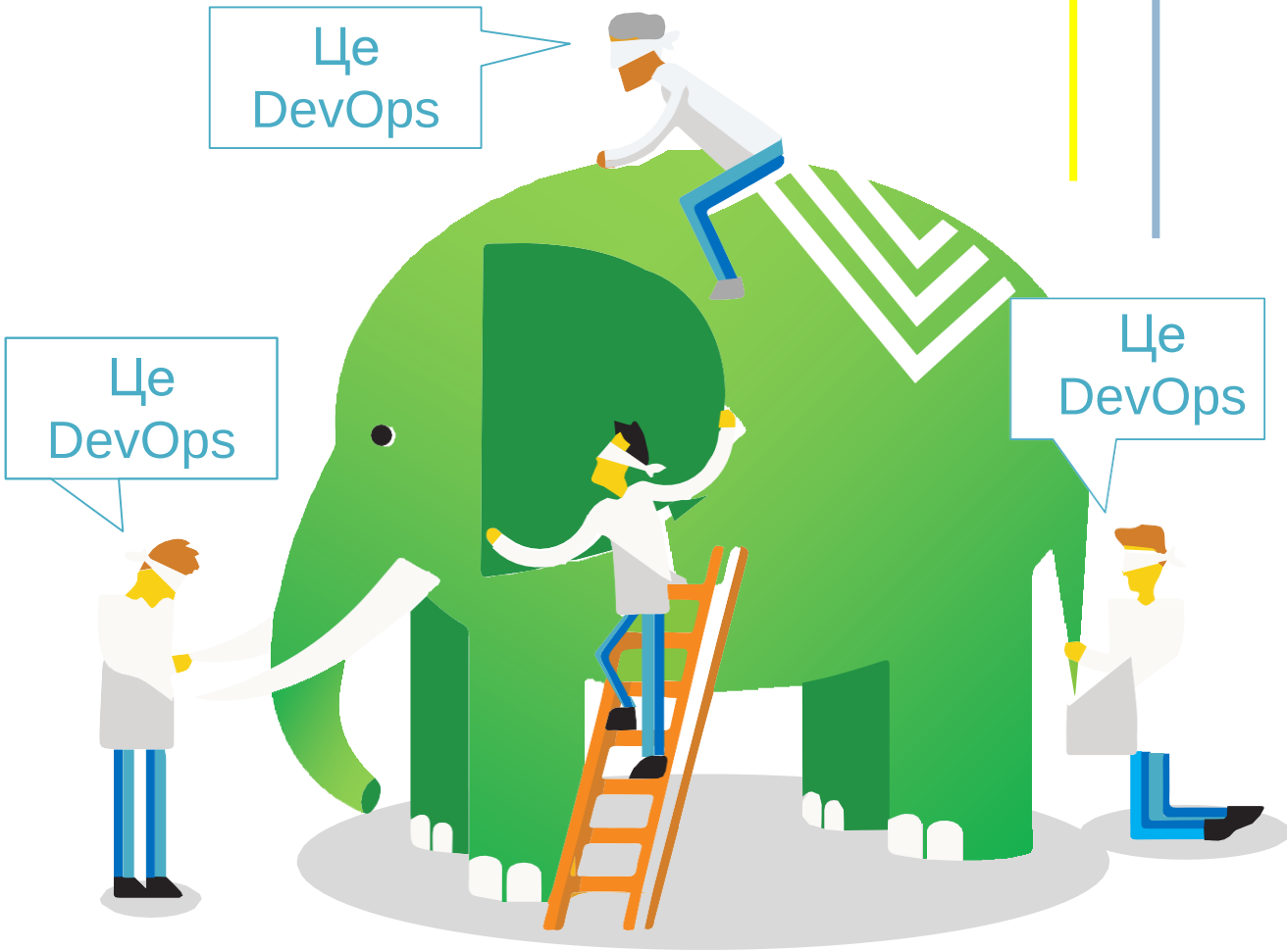
- 16 лекцій
- 14 лабораторних робіт
- Екзамен

Що таке DevOps

DevOps* – ц е інженерна культура, практики та інструменти, які направлені на скорочення релізного циклу, підвищення ефективності та забезпечення можливості випуска релізу в будь-який момент шляхом:

- інтеграції IT Development та IT Operations;
- створення єдиної команди;
- автоматизації рутинних операцій;
- частої перевірки коду

! META: скоротити T2M* та підвищити гнучкість впровадження (release anytime) без втрати якості та надійсності



. *DevOps – акронім від англ. development та operations*
. *T2M (time-to-market) – строк від початку розробки продукту до його виходу на ринок*

Упередження щодо DevOps

Хтось з IT: А що, DevOps буде замість Agile?

Хтось з IT: DevOps та Waterfall несумісні!

Менеджер IT: Ми в ці вихідні не працюємо, у вас буде час прийти та встановити мені DevOps.

Dev: DevOps – це задача адміністраторів.

Dev: У мене установка на середовища розробки йде однією кнопкою.
Язробив DevOps!

Dev: Мені DevOps не потрібен. Ми всі генії розробки, а Вася робить збірку навіть під час сну.
Питання: *Що буде, якщо кількість задач збільшиться, в вашу команду додатуться нові розробники, а Вася піде в декрет?*
Dev: Це проблема нових розробників, а Вася …



Ops: Ви розробіть процес розгортання та покажіть, а ми надамо свої зауваження.

Ops: У нас і так все добре та надійно. Не потрібні нам нові інструменти. ***Питання:*** *Що буде, якщо не один реліз в місяць, а два в тиждень?*
Ops: Терміново збільшуємо штат!

Ops: Ну добре, впровадили DevOps. Де моя одна кнопка?

Менеджер IT: Потрібно активно попрацювати три місяці, зробимо DevOps т а тоді розслабимося

Питання: *Вас не бентежить слово Continuous в назвах практик DevOps?*

Ops: Е-е-е …

Основні компоненти DevOps



Тільки змінив всі чотири складові, можна впровадити DevOps!

DevOps принципи

Culture	• Розвиток інженерної культури – від розробника до інженера • Від відповідальності за етап– до відповідальності за результат
Automation	• Реліз в будь-який час • Прийняття змін та зменшення ризиків
Measurement	• Безперервне покращення, яке базується на метриках та реальних даних
Sharing	• Колаборація та відкрита комунікація • Постійний зворотній зв’язок «справа наліво»

Основні практики DevOps, пов’язані в циклічний автоматизований процес

Безперевне розгортання (Continuous Deployment)

Автоматичне розгортання на середовищах аж до промислового

Безперервна доставка (Continuous Delivery)

Автоматичне розгортання та тестування

Безперервна інтеграція (CI*)

Автоматична збірка, запуск модульних тестів та статичного аналізу

1 Кодування

2 Модульне тестування

3 Інспекція кода, статичний аналіз коду та ІБ

4 Збірка та розгортання на середовищах розробки

5 Розгортання на СТ

6 Тестування на СТ

7 Розгортання на ІФТ, НТ, ІБ

8 Тестуваня на ФТ, НТ, ІБ

9 Реєстрація в сховище дистрибутивів організації

10 Розгортання та проведення ПВ / Hotfix

11 Розгортання на ПРОД

- **Підвищення якості коду за рахунок:**
  - практик модульного тестування (unit testing)
  - іспекції коду(code review) та статичного аналізу коду на відповідність кращим світовим стандартам кодування та безпеки
- **Скорочення строків розробки за рахунок:**
  - прискорення циклу зворотнього зв’язку
  - збільшення частоти збірок (мінімум 1 раз на день)

- **Підвищення якості тестування за рахунок:**
  - впровадження практик автоматизованого тестування - testing automation
  - впровадження динамічного аналізу застосунків на вразливість
- **Скорочення часу виведення в ПРОД за рахунок:**
  - автоматизації розгортання дистрибутивів на середовища
  - динамічного виділення інфраструктури, infrastructure as code

- **Підвищення частоти введення релізів в ПРОД за рахунок:**
  - впровадження практик автоматизації розгортання

*Continuous Integration

Практики автоматизованого тестування Continuous Testing



Види автоматизованого тестування	Що це таке?	Хто розробляє та проводить тести?
Модульне тестування – unit testing	Перевірка працездатності частини коду (функції, класу і т.д.). Тести пишуться на мові програмування з використанням спеціальних фреймворків	Інженер Agile - команди
Статичний аналіз коду та ІБ	Перевірка коду на відповідність міжнародним стандартам мов програмування та написання безпечного коду за допомогою спеціальних інструментів.	
Системне тестування	Перевірка працездатності ізольованої частини функціоналу системи (без інтеграції).	
Інтеграційно-функціональне тестування	Перевірка працездатності наскрізних бізнес-операцій між різними системами для нового та для раніше розробленого функціоналу.	Інженер тестування
Навантажувальне тестування	Перевірка працездатності системи при одночасній роботі більшої кількості користувачів. Виконується тільки для високонавантажених систем та бізнес-операцій	
Динамічне тестування застосунку на безпеку	Перевірка системи на вразливості (підвищення привілеїв, виконання довільного коду, sql injection,...) та «закладки»	

Циклічний автоматизований процес на середовищах організації

- Безшовний наскрізний процес виробництва ПЗ
- Максимальна автоматизація на кожному етапі процесу
- Автоматизована збірка та розгортання дистрибутива
- Автоматичне тестування, автоматичний аналіз якості коду, перевірки коду та застосунків на ІБ
- Автоматизоване управлення інфраструктурою, інфраструктура як код, динамічне виділення інфраструктури з хмари

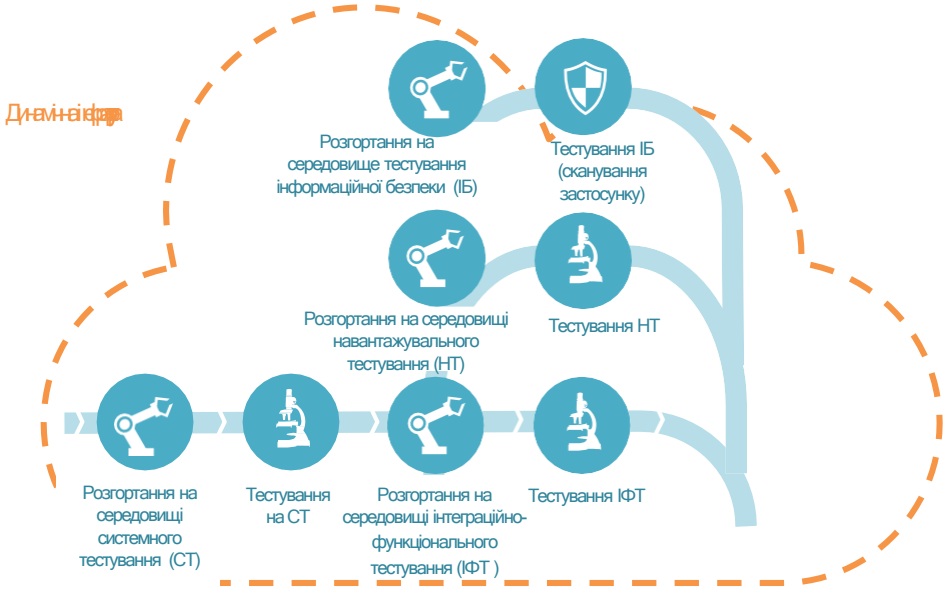


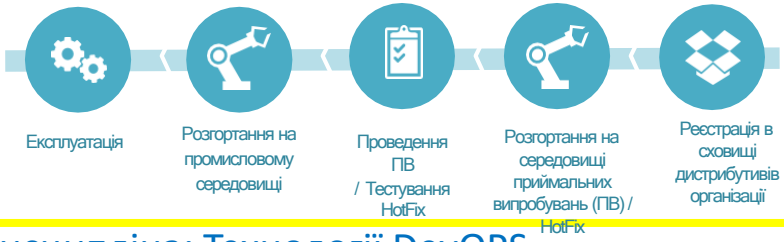
Основні практики DevOps



Безперервна інтеграція (CI, Continuous Integration) – це практика розробки програмного забезпечення, яка полягає у виконанні частих автоматизованих збірок для якнайшвидшого виявлення помилок. Це дозволяє бути впевненим у тому, що код перебуває у робочому стані. Перехід до безперервної інтеграції дозволяє знизити трудомісткість інтеграції змін коду з існуючою кодобазой і зробити її більш передбачуваною за рахунок найбільш раннього виявлення та усунення помилок та протиріч, зробити процес розробки прозорим для всіх учасників команди.

Безперервна доставка (CD_L , Continuous delivery) – практика розробки програмного забезпечення, що гарантує те, що програмне забезпечення завжди готове до розгортання промислової експлуатації. Практика включає автоматизацію розгортання на тестові середовища, автоматизацію тестування успішності розгортання на рівні адміністраторів (технічні та функціональні тести розгортання), а також інтеграцію з процесами автоматизованого функціонального, навантажувального тестування та динамічного тестування на безпеку.





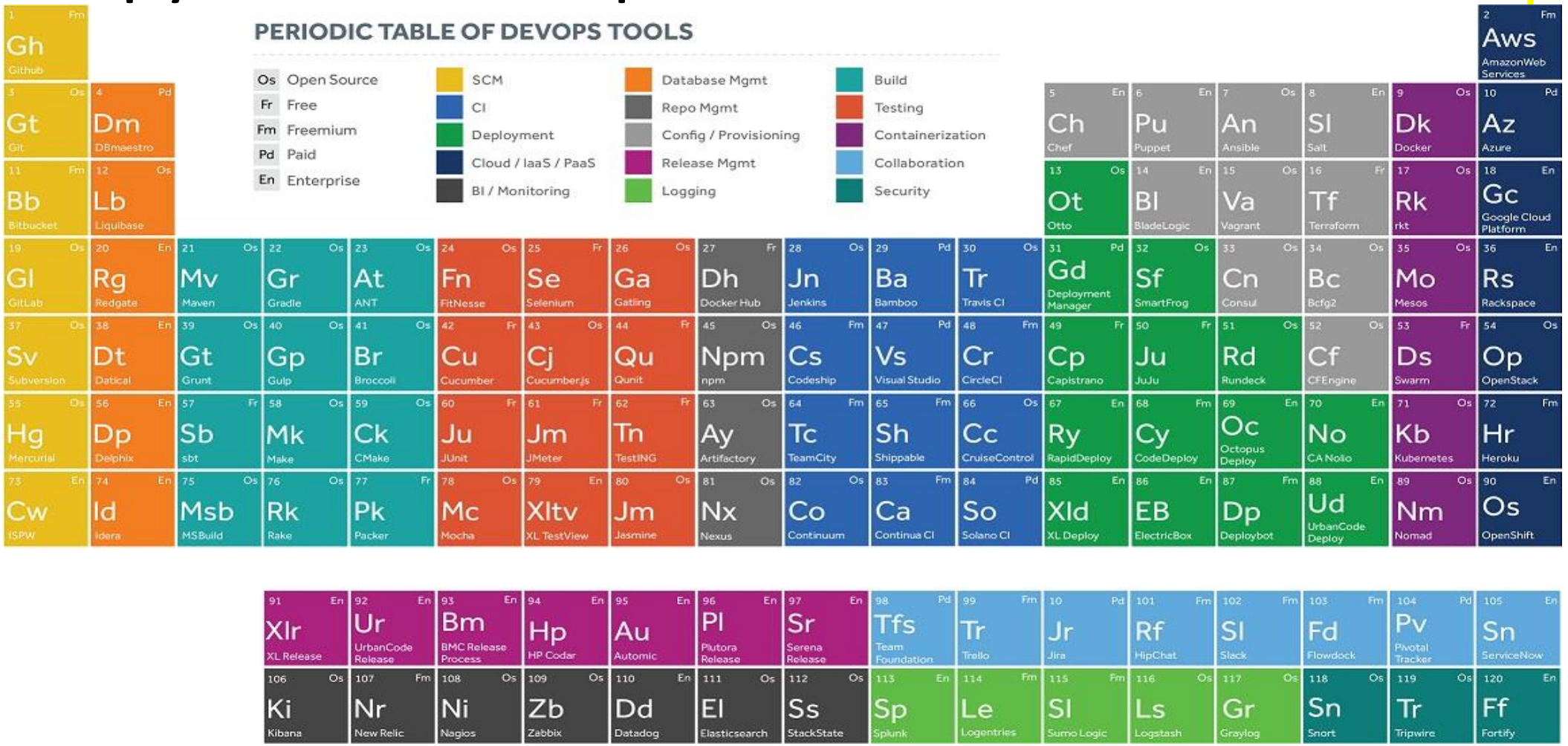
Безперервне розгортання (CD_P , Continuous deployment) – практика розробки програмного забезпечення, спрямована на повну автоматизацію доставки від середовища розробки у промислове середовище. Є подальшим розвитком процесу безперервної доставки. В умовах організації додатково зачіпає процес реєстрації у сховищі дистрибутивів організації та автоматизацію приймальних випробувань. Характеризується тісною взаємодією розробки та експлуатації.

Проблеми, що вирішуються



Існуючі проблеми виробництва	Вирішення за допомогою DevOps	
1 Через відсутність загальних стандартів ведення розробки (патерни програмування, нотації тощо) та централізованого зберігання коду трапляються конфлікти версій, пишеться неоптимальний код, знижується ефективність розробки.	VC	Код зберігається централізовано, будь-яка зміна зберігається з новою версією. Розробка ведеться згідно зі стандартами, адаптованими до команди.
2 Розробка не виконує інтеграційні тести, тому помилки інтеграції виявляються пізніх етапах -до ПВ.	AT	При кожній збірці виконуються швидкі smoke тести, що перевіряють інтеграцію “на заглушках” та емуляторах, помилки виявляються відразу при складанні.
3 Ручне керування середовищами та розгортанням призводить до тривалого очікування стендів командами (до кількох місяців) та збільшує кількість помилок при налаштуванні стендів.	AD IaaS DI	Виділення середовища відбувається за хвилини з хмари організації. Розгортання та конфігурування автоматизовано.
4 Великий обсяг ручного тестування, відсутність модульних та швидких “smoke” тестів збільшує час тестування або скорочує обсяг тестового покриття.	AT	Весь код покритий модульними тестами, тестування є максимально автоматизованим.
5 Перевірка якості та безпеки коду при складанні не є обов’язковою. Тому в результаті можемо отримати небезпечний (з уразливістю) та неоптимальний код (збільшення навантаження на супровід).	ACR PrCR IS JaaS	Код і готовий застосунок перевіряються на вразливості та на якість - як в Apple і Google всі події централізовано журналуються.

На ринку присутні десятки інструментів DevOps

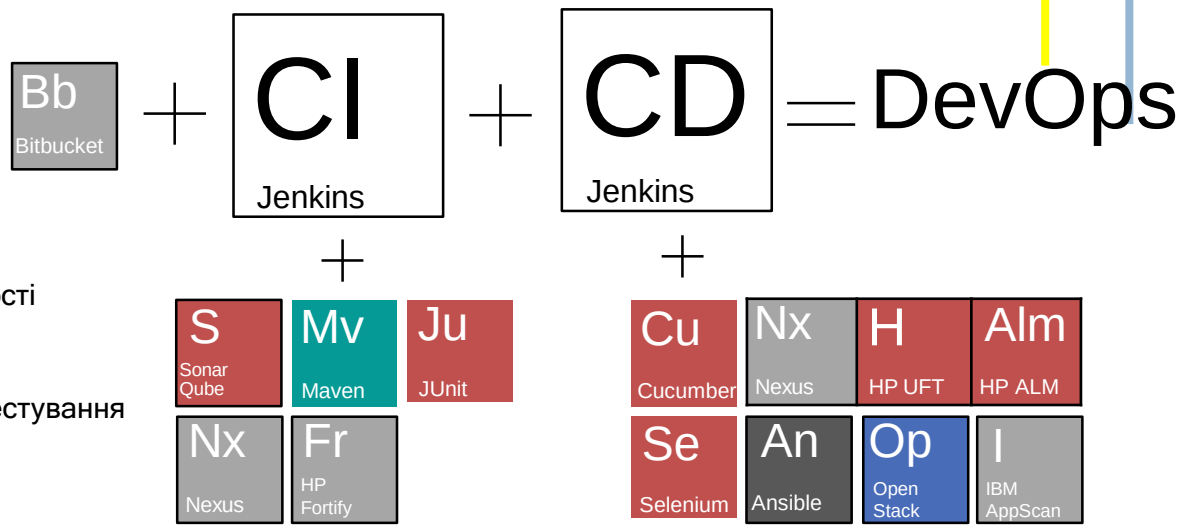


Джерело: https://xebialabs.com/periodic-table-of-devops-tools/

Інструменти: затверджені та ті, що обираються командами

Затверджені інструменти:

- **Atlassian Bitbucket** – система версіонного зберігання коду
- **Maven** – інструмент авто-збірки
- **JUnit** – бібліотека для модульного тестування Java застосунків
- **Jenkins** – інструмент управління процессом збірки, розгортання та налаштування
- **Sonar, HP Fortify, IBM AppScan** – перевірка коду на якість та вразливості
- **HP ALM** – інструмент управління процесом тестування
- **HP UFT, Selenium, Cucumber, HP PC** – автотести, навантажувальне тестування
- **Ansible** – управління конфігураціями та налаштуванням середовищ
- **Nexus** – управлення зберіганням дистрибутивів
- **OpenStack** – віртуалізація інфраструктури, приватна хмара



☐ - Цільовий інструмент, затверджений/ рекомендований до використання

Інструменти, що обираються командами:

Однак є інструменти, тісно пов'язані з певною технологією. Такі інструменти не затверджуються на архітектурній раді, їх вибір здійснюється робочою групою:

- Командою розробки
- ЦЕ процесів виробництва (ЦЕ DevOps)
- Командою сервіса CI або CD
- Прикладними та системними адміністраторами

Чому нам важливо мати стандарти у виборі інструментів DevOps?

- Централізований супровід інструментів дозволяє командам зосередитись на розробці
- Надання централізованого сервісу 24x7
- Об’єктивний вибір інструментів з урахуванням критеріїв вартості (в т.ч. вартість експлуатації), ефективності та надійності та відповідність інструментів принципам прозорості
- Наявність безлічі інструментів*, що виконують однакову функцію, створює «вузьке місце» на стороні Ops, що суперечить принципам DevOps.
- До централізованих інструментів додається централізований моніторинг, який дозволяє простежити весь шлях релізу від розробки до ПРОДу.

DevOps Toolchain

Безперервна інтеграція

1. Кодування



Розробник програмує код і пише модульні тести до цього коду

Інструменти: Bitbucket JUnit

2. Управління збіркою



Після збереження коду в **Bitbucket** запускається workflow процесс збірки в **Jenkins** . **Jenkins** запускає модульні тести, статичний аналіз якості коду та ІБ. **Ansible** здійснює автоматизоване розгортання дистрибутива на середовище розробки.

Інструменти: Nexus Bitbucket Jenkins ANSIBLE

Інструменти збірки: FORTIFY sonarqube

Безперервна доставка

3. Підготовка середовища



Після складання дистрибутива запускається workflow процесу розгортання в **Jenkins** , відбувається (опціонально) виділення тестових середовищ, готуються тестові набори даних, здійснюється автоматичне конфігурування середовищ через **Ansible**.

Інструменти: Jenkins ANSIBLE Nexus

4. Управління розгортанням



Ansible розгортає дистрибутив на тестові середовища, виконуються технічні та функціональні тести розгортання

Інструменти: Jenkins ANSIBLE Nexus

5. Управління тестуванням



Jenkins викликає **HP ALM** і запускається набір тестів як автоматизованих (API, GUI, інтеграційних, безпеки тощо), так і «ручних»

Інструменти: hp ALM Unified Functional Testing (UFT) AppScan IBM Security

Моніторинг та зворотній зв'язок



Механізм оповіщень та моніторингу гарантує швидкий цикл зворотного зв'язку. Доступний для всіх етапів

Інструменти: elastic QlikView

Безперервне розгортання

6. ПВ



У разі успішного проходження випробувань приймається рішення (людина) про розгортання дистрибутива на ПРОД середовище

7. Розгортання на ПРОД



Відбувається автоматизоване розгортання дистрибутива на середовище ПРОД. У разі помилок - виконується автоматичний відкат (Rollback)

Інструменти: Jenkins ANSIBLE Nexus

Нова роль – DevOps інженер



DevOps інженер – є експертом з практик DevOps, володіє інструментами CI і CD, вибудовує Pipeline збірки та розгортання.

- Налаштовує процес CI на централізованому сервісі, інтегрує його з GitFlow та кодом розгортання на середовищах розробки.
- Розробляє процес постачання ПЗ на цільових інструментах CD для всіх середовищ (розробки, тестування та експлуатації).
- Забезпечує однаковість процесів доставки та вибудовує їх у єдиний безшовний процес доставки.
- Керує та забезпечує процес поширення змін тестових середовищ для середовищ розробки та тестування (за межами обов'язків середовища організації).
- Надає супровід прикладним та системним адміністраторам середовищ організації
- Має інструменти централізованого сервісу CI і CD, а також командні інструменти, в т.ч. з юніт-тестування та збірк.
- Має знання технічної та інтеграційної архітектури, механізмів переносів налаштувань між середовищами.
- Має базові навички адміністрування прикладного і системного ПЗ.

Робота DevOps Toolchain

